

AdaMMS: Model Merging for Heterogeneous Multimodal Large Language Models with Unsupervised Coefficient Optimization

Yiyang Du^{*,1}, Xiaochen Wang^{*,3,4}, Chi Chen^{*,1}, Jiabo Ye⁵, Yiru Wang⁸,
Peng Li^{✉,2,6}, Ming Yan⁵, Ji Zhang⁵, Fei Huang⁵, Zhifang Sui³, Maosong Sun¹, Yang Liu^{✉,1,2,6,7}

¹Dept. of Comp. Sci. & Tech., Institute for AI, Tsinghua University, Beijing, China

²Institute for AI Industry Research (AIR), Tsinghua University, Beijing, China

³State Key Laboratory of Multimedia Information Processing, Peking University, Beijing, China

⁴School of Software Microelectronics, Peking University, Beijing, China

⁵Institute of Intelligent Computing, Alibaba Group

⁶Shanghai Artificial Intelligence Laboratory, Shanghai, China

⁷Jiangsu Collaborative Innovation Center for Language Competence, Jiangsu, China

⁸ModelTC Open Source Organization, Beijing, China

Abstract

Recently, model merging methods have demonstrated powerful strengths in combining abilities on various tasks from multiple Large Language Models (LLMs). While previous model merging methods mainly focus on merging homogeneous models with identical architecture, they meet challenges when dealing with Multimodal Large Language Models (MLLMs) with inherent heterogeneous property, including differences in model architecture and the asymmetry in the parameter space. In this work, we propose AdaMMS¹, a novel model merging method tailored for heterogeneous MLLMs. Our method tackles the challenges in three steps: mapping, merging and searching. Specifically, we first design mapping function between models to apply model merging on MLLMs with different architecture. Then we apply linear interpolation on model weights to actively adapt the asymmetry in the heterogeneous MLLMs. Finally in the hyper-parameter searching step, we propose an unsupervised hyper-parameter selection method for model merging. As the first model merging method capable of merging heterogeneous MLLMs without labeled data, extensive experiments on various model combinations demonstrated that AdaMMS outperforms previous model merging methods on various vision-language benchmarks.²

^{*}Equal contribution.

[✉]Corresponding authors: Peng Li (lipeng@air.tsinghua.edu.cn) and Yang Liu (liuyang2011@tsinghua.edu.cn).

¹AdaMMS represents **Ad**aptive **M**apping, **M**erging, and **S**earching.

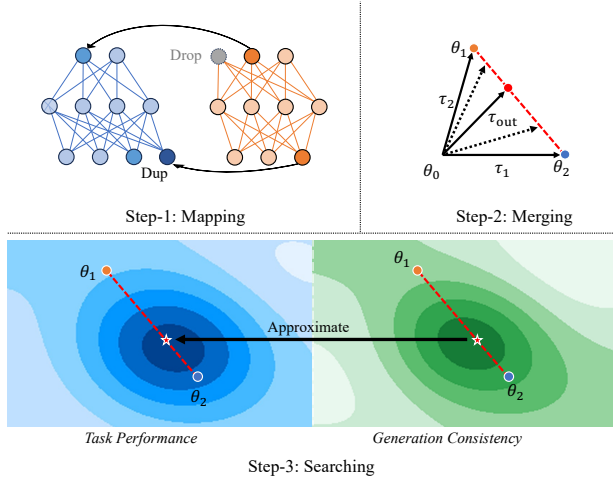
²Code at <https://github.com/THUNLP-MT/AdaMMS>.

1. Introduction

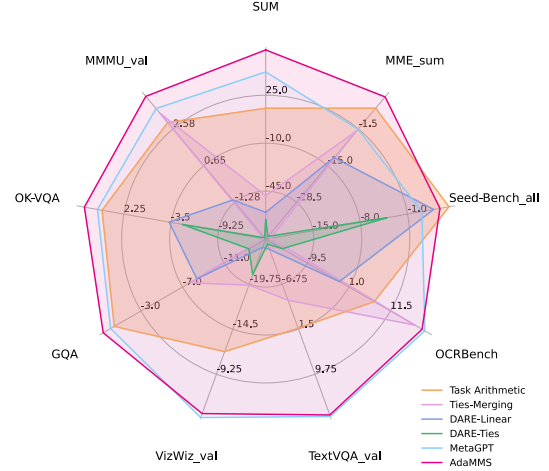
Model merging [10] has gained increasing popularity in the field of large language models (LLMs) [10, 26, 29, 32]. This approach typically involves combining the parameters of two models with the same architecture, creating a new model without requiring additional training [10]. It has proven to be an efficient method for developing models that integrate the abilities of multiple existing models [26, 29, 32], avoiding the need for extensive data and computational resources, and has been widely adopted in building powerful LLMs [4, 6].

Despite its popularity with LLMs, model merging has yet to be widely adopted for multimodal large language models (MLLMs). Some recent studies have explored the application of model merging to MLLMs [1, 19] but either prioritize extending multimodal capabilities over enhancing the performance of existing models [1] or still require additional training after merging [19]. The primary obstacle in applying model merging to MLLMs lies in the heterogeneous nature of these models [8, 12, 14, 24, 28]. This heterogeneity arises from modifications to the transformer architectures [21] within their language models [8, 28], as well as differences in their choice of modality-specific encoders and tokenizers [8, 24]. Consequently, a significant challenge in merging MLLMs is that the process cannot be directly applied to models with different architectures, as their weights are not isomorphic.

Recent efforts like FuseLLM [22] and FuseChat [23] have explored fusing the capabilities of heterogeneous LLMs by merging their generative distributions. Theoretically, these methods could also be applied to MLLMs.



(a) AdaMMS overview.



(b) Gains from different merging methods.

Figure 1. (a) Illustration of three steps in AdaMMS: Step-1, mapping MLLMs with different model architecture; Step-2, merging MLLMs with linear interpolation; Step-3, searching for optimal merging hyper-parameter by approximate task performance through generation consistency without labeled data. (b) The gain performance of AdaMMS on a broad range of multimodal tasks in comparison with existing merging approaches. Gain refers to the improvement obtained by subtracting the average result from the result of the fused model on a certain task. The result here is the average of the gains from the two MLLM pairs merging.

However, they rely on supervised continued training, which incurs significant computational costs and fails to address scenarios where labeled data is unavailable. For example, FuseLLM requires a training dataset with a total of 1.8 billion tokens and 33 hours of training time. This underscores the need for an unsupervised model merging technique to effectively integrate heterogeneous MLLMs.

In this work, we address the challenges of merging heterogeneous MLLMs by introducing a novel model merging strategy named AdaMMS, as illustrated in Figure 1. First, to enable model merging across heterogeneous MLLMs with differing architectures, we design a parameter mapping framework. Specifically, we focus on the scenario where MLLMs have different language model architectures due to variations in transformer block duplications [8, 25, 28]. This mapping aligns corresponding components across models, enabling merging operations even with structural differences. Next, we apply adaptive linear interpolation on the mapped parameters during merging. By adjusting the interpolation coefficient, AdaMMS optimizes performance adaptively across different tasks. This coefficient adjustment is then optimized through an unsupervised procedure in the following step. Finally, we introduce an **unsupervised** hyperparameter selection method to determine the interpolation coefficient for each task. This approach leverages response consistency across candidate models as a performance estimate, based on our novel insight that model performance correlates with generation consistency. In addition to strong empirical support, we

provide a theoretical analysis to explain this insight.

Extensive experiments have demonstrated the effectiveness of our proposed AdaMMS in merging MLLMs. Our main experiments are conducted on two pairs of heterogeneous MLLMs, one of them is based on Qwen [27] architecture, another pair is based on LLaMA [20] architecture. Both experiments shows that our model can effectively combine the capabilities of heterogeneous MLLMs by enabling model merging on models with different architecture. The experiments also demonstrated that the merging strategy of AdaMMS, accompanied with our unsupervised hyper-parameter selection method, outperforms previous model merging methods on various vision-language tasks. We also conducted experiments to show that the unsupervised hyper-parameter selection method can be performed with fewer unlabeled data without harming its performance, which shows the robustness of the method and further decrease the data requirements of AdaMMS.

Our main contributions are as follows:

- We introduce a novel model-merging strategy, AdaMMS, designed to address the challenges of merging heterogeneous MLLMs. By defining a parameter mapping between different models, AdaMMS facilitates model merging techniques even when architectural differences exist.
- We propose an *unsupervised* hyperparameter selection method inspired by the observation that model performance can be effectively estimated through generation consistency—measured by the variation in generated responses. Unlike previous approaches requiring labeled

data, this method eliminates the dependency on annotations and can be applied to a small subset of 100 target sample without sacrificing effectiveness.

- Comprehensive experiments conducted on various model pairs demonstrate the effectiveness of our approach. Specifically, evaluations on Qwen-based and LLaVA-based heterogeneous MLLM pairs show that AdaMMS successfully combines capabilities from distinct MLLMs. Across various vision-language tasks, AdaMMS consistently outperforms baseline strategies, achieving superior overall performance.

2. Related Work

2.1. Model Merging

Recent researches on model merging techniques have contributed to building more capable models by providing an efficient approach to combine abilities on various tasks from different models that requires less data and compute. Some studies focus on merging homogeneous models with identical architecture, while others focus on tackle the challenge on heterogeneous models which have different architectures. In merging homogeneous models, Task Arithmetic [10] proposes the concept of task vectors, which subtracts fine-tuned weights from pre-train weights to obtain task-related weight difference as the object of merging. Ties-Merging [26] and DARE [29] further improve the performance by mitigating parameter interference during the merging process through parameter pruning and conflict resolving. MetaGPT [32] scales the task vectors with task-agnostic coefficients in closed-form by separating data term and scaling coefficients in the optimization objective. Although these methods improves the performance of the merged models, they cannot be directly applied on models with architecture difference. In fusing heterogeneous models, DAMC [1] employs parameter decoupling and adaptive adjustment to enhance model merging strategies for fusing modalities on MLLMs with different modality encoders, but this work still focus on merging identical language model architecture. To consolidate LLMs with different architectures, FuseLLM [22] and FuseChat [23] applies token alignment and model fusion strategies with knowledge distillation before continue training the model, but they need labeled data and computation resources for continue training. In fact, the majority of previous works on model merging requires labeled data for validation search or supervised training [1, 10, 22, 23, 26, 29]. In this work, we eliminate the need of labeled data by leveraging our unsupervised hyper-parameter selection method, and enable model merging strategies to be applied on heterogeneous MLLMs with architecture differences.

2.2. Multimodal Large Language Models

As large language models demonstrate huge success in obtaining great abilities in general, recent researches on MLLMs have successfully appending multimodal processing and generation ability on LLMs, especially on the vision modality [2, 12, 14, 24, 25, 28]. However, these models often adapts unique modifications on language model architecture, resulting in a set of heterogeneous MLLMs, which prevent model merging methods to be applied on them. Specifically, there are two levels of architecture differences among MLLMs. First, two MLLMs may be designed from different pre-trained language model. For example, Qwen2-VL [24] and LLaVA-OneVision-Qwen [12] are designed from Qwen2 [27], while LLaVA [14], mPLUG-Owl2 [28], CogVLM [25] and ShareGPT4V [2] are designed following the LLaMA [20] architecture. Second, two MLLMs developed from the same pre-trained language model can still be heterogeneous because they are designed with different modifications on the language model. For example, although CogVLM and mPLUG-Owl2 are both developed from LLaMA architecture, CogVLM adapts visual experts by duplicating query, key and value weights in attention head, while mPLUG-Owl2 is designed to duplicate key, value, and layer norm weights instead. The first level of differences is hard to merge, since model merging applies to parameters that are trained from the same pre-training weights [10]. In this work, we tackle the second level of architecture differences via our proposed AdaMMS method.

3. Method

To tackle the heterogeneous challenges in merging MLLMs, we propose a novel model merging method named AdaMMS. As shown in Figure 1(a), it involves three steps: mapping, merging and searching. In the mapping step, we define a mapping function that enables the merging of parameters from different architectures. Next, in the merging step, we apply linear interpolation to adaptively optimize the performance on specific downstream tasks. Finally, in the searching step, we design a unsupervised hyper-parameter selection method for choosing linear interpolation coefficient during merging. This method is based on our novel discovery that the model performance in the parameter space can be approximated by the difference among model responses without the need of labeled data.

3.1. Mapping

To merge the parameters of two heterogeneous models M_1 and M_2 into M_1 's architecture, we need to align their parameters by defining a mapping f that maps each parameter θ_1 in M_1 to its corresponding parameter θ_2 in M_2 (or ϕ if there is no such corresponding parameter). As previously discussed in Section 2.2, we only tackle the hetero-

geneous MLLMs that are designed from same pre-trained language model architecture, but adapts different modifications on model structure.

The principle of designing the mapping f is that for the shared weights between the models (e.g. the weights in the pre-trained model), we can map them directly, and for additional weights in M_1 , we map it to its original corresponding weight in M_2 if it is a duplicated multimodal parameter in M_1 's specific design, otherwise we map ϕ with it and apply no operation later in merging. In this way, we leverage the additional weights as much as possible without mapping irrelevant parameters together.

3.2. Merging

We follow the paradigm in Task Arithmetic [10] to apply merging operation on task vectors and apply linear interpolation on them. Task vectors are defined as the finetuned parameters subtracted by pre-train weights: $\tau_i = \theta_i - \theta_0$ ($i = 1, 2$), where θ_1 and θ_2 is the models to be merged, and θ_0 is their common initialization point (e.g. the shared pre-trained weights for two different finetuned model). Linear interpolation offers the availability to directly control the tendency between the two models alongside its simplicity. This allows us to actively adapt to different downstream tasks, as different downstream tasks often requires different combination or tendency on the two models for the best performance. Linear interpolation on task vectors can be simply formatted as: $\theta_{out} = \theta_0 + (1 - \alpha)\tau_1 + \alpha\tau_2$, where α is the linear interpolation coefficient. This is equivalent to: $\theta_{out} = (1 - \alpha)\theta_1 + \alpha\theta_2$. Thus, leveraging our mapping function f in Section 3.1, we can apply our merging operation on heterogeneous MLLMs as follow:

$$\theta_{out}^i = \begin{cases} \theta_1^i, & \text{if } f(\theta_1^i) = \phi \\ (1 - \alpha)\theta_1^i + \alpha f(\theta_1^i), & \text{otherwise} \end{cases} \quad (1)$$

Note that $f(\theta_1)$ is the parameter in M_2 that corresponds to θ_1 , according to the mapping function f . Now we can define the merging process of two model parameters $\Theta_1 = \{\theta_1^i\}$ and $\Theta_2 = \{\theta_2^j\}$ accordingly:

$$\text{Merge}(\Theta_1, \Theta_2; f; \alpha) = \{\theta_{out}^i\} \quad (2)$$

3.3. Searching

Consider the base model, which is defined as the architecture that will be used after the merging process, containing N scalar weight elements, then the merging process can be seen as the operation on a N -dimensional vector space $\mathbb{R}^N$, where the merging strategy F transform two input points as initial parameters Θ_1 and Θ_2 to the merged parameters Θ_{out} . For simplicity, we consider the case when the merging strategy F takes one hyper-parameter α (linear interpolation coefficient).

$$\Theta_{out} = F(\Theta_1, \Theta_2; \alpha), \quad \Theta_1, \Theta_2, \Theta_{out} \in \mathbb{R}^N$$

Given the inputs t_i on a downstream task, this creates a landscape on $\mathbb{R}^N$ that each model parameter corresponds to the model performance S_{t_i} on the inputs.

$$S_{t_i}(\Theta_{out}) = S_{t_i, F}(\Theta_1, \Theta_2, \alpha)$$

Therefore, the goal of the hyper-parameter searching is to find the best α that maximize the merged model's performance on the tasks. For simplicity, we omit the F in the index.

$$\alpha^* = \underset{\alpha}{\operatorname{argmax}}(S_{t_i}(\Theta_1, \Theta_2, \alpha))$$

Note that as the landscape varies greatly in different tasks, the best α may be different as well.

Previous model merging methods mainly relies on a validation set to search for the best hyper-parameter α with supervised searching. Formally, they use the best α in validation set $\hat{t}_i$ to approximate the best α in test set t_i .

$$\hat{\alpha} = \underset{\alpha}{\operatorname{argmax}}(S_{\hat{t}_i}(\Theta_1, \Theta_2, \alpha)) \approx \underset{\alpha}{\operatorname{argmax}}(S_{t_i}(\Theta_1, \Theta_2, \alpha))$$

However, this supervised way of searching has certain disadvantages: (1) labeled data with ground truth is hard to collect in some scenarios, and (2) the distribution shift between the validation set and the test set, even on the same task, will interfere the selection of the best hyper-parameter α^* . To get rid of these drawbacks, we propose an unsupervised hyper-parameter selection method through a performance estimation metric that requires no labeled data.

Specifically, we discover that the difference of generated responses between two adjacent α candidates can be used to estimate the model performance, and the best α can be approximated by the one with the lowest adjacent difference. As shown in Figure 2, the trend of model performance is similar with its generation consistency which is measured by response differences, and the α with the highest generation consistency match the α with the highest performance. Formally, let α^- and α^+ be adjacent candidates on both sides of α , respectively, and $D_{t_i}(\alpha; \alpha^-, \alpha^+)$ denoting the difference of generated responses between , we take:

$$\bar{\alpha} = \underset{\alpha}{\operatorname{argmin}}(D_{t_i}(\alpha; \alpha^-, \alpha^+)) \approx \underset{\alpha}{\operatorname{argmax}}(S_{t_i}(\Theta_1, \Theta_2, \alpha))$$

as the approximation of the best choice for α . This eliminates the need of labeled data with ground truth, and avoids the data distribution shift between validation set and test set.

The discovery indicates that near the best performance point, the model response tends to be more stable. This can be explained via a convex hypothesis. Suppose the landscape on given task t_i is convex in a subspace of the parameter space that covers the candidate results of merged models, that is, for any $\lambda \in [0, 1]$ we have:

$$S_{t_i}(\lambda\Theta_1 + (1 - \lambda)\Theta_2) \geq S_{t_i}(\lambda\Theta_1) + S_{t_i}((1 - \lambda)\Theta_2)$$

This guarantee that the optimum is attained where the gra-

dient vanishes:

$$\nabla_{\Theta} S_{t_i}(\Theta^*) = 0$$

where $\Theta^* \in \mathbb{R}^N$ is the optimal parameter value. In a convex function, the Hessian $H(\Theta^*) = \nabla_{\Theta}^2 S_{t_i}(\Theta^*)$ at the optimum Θ^* is positive semi-definite, which implies local stability in a neighborhood around Θ^* . The stability can be characterized by the second-order Taylor expansion:

$$S_{t_i}(\Theta) \approx S_{t_i}(\Theta^*) + \frac{1}{2}(\Theta - \Theta^*)^\top H(\Theta^*)(\Theta - \Theta^*)$$

Since $H(\Theta^*) \geq 0$, small deviations from Θ^* will result in small increases in the performance, ensuring relative stability on the landscape. Although the convex hypothesis is ideal, we confirm the effectiveness of our unsupervised hyper-parameter selection method in various experiments. Proof in Appendix F further shows the relationship between generation consistency and model performance.

We also find that while the landscape often changes between the validation and test sets and across different tasks, it remains consistent between the full test set and a small subset. This suggests that we can perform unsupervised hyper-parameter selection on a smaller subset of the data $\bar{t}_i$ without compromising its accuracy.

$$\bar{\alpha} = \underset{\alpha}{\operatorname{argmin}}(D_{\bar{t}_i}(\alpha; \alpha^-, \alpha^+)) \approx \underset{\alpha}{\operatorname{argmax}}(S_{t_i}(\theta_1, \theta_2, \alpha))$$

In conclusion, the process of AdaMMS is described in Algorithm 1.

4. Experiment

4.1. Baselines

Previous model merging methods cannot be directly applied to heterogeneous MLLMs with architecture difference, and our mapping method enables the fusion of heterogeneous models by transforming them into a homogeneous parameter space. Therefore, all the baseline experiments are conducted under the precondition of the mapping step of our proposed method. We consider the following model merging methods as our baselines:

- **Task Arithmetic** [10] introduces the idea of task vectors and integrates them into the original pre-trained model for multi-task learning.
- **Ties-Merging** [26] further addresses interferences in Task Arithmetic by removing unnecessary parameters from Task Arithmetic [10]. This process eliminates redundant parameters and resolves symbol conflicts through Trim, Elect Sign, and Disjoint Merge steps.
- **DARE** [29] tackles the parameter conflict problem in model merging by applying a drop and rescale operation before merging model weights. There are two variants of DARE: **Dare-Linear** and **Dare-Ties**, which perform different merging strategies after the drop and rescale operation. Dare-Linear performs linear interpolation, and

Algorithm 1 AdaMMS Procedure

Input: Original MLLMs M_1, M_2 , their parameters Θ_1, Θ_2 respectively, a subset of test inputs $\bar{t}$, and the candidates of the hyper-parameter $\{\alpha_n\}$

Output: Merged parameters Θ_{out} on M_1 's architecture

- 1: Define a mapping f that maps each parameter θ_1 in M_1 architecture with its corresponding parameter θ_2 in M_2 architecture (if exists) ▷ Section 3.1
 - 2: Define a process $\text{Generate}(\Theta, \bar{t})$ that returns the generation responses G of model with parameters Θ on inputs $\bar{t}$
 - 3: Define a function $\text{DiffCnt}(G_i, G_j)$ that counts the number of corresponding elements in G_i and G_j that do not exactly match
 - 4: **for** $i = 1$ to n **do**
 - 5: **for each** hyper-parameter candidate α_i in $\{\alpha_n\}$ **do**
 - 6: $\Theta_{\text{cand}}^i \leftarrow \text{Merge}(\Theta_1, \Theta_2; f; \alpha_i)$ ▷ Equation (2)
 - 7: $G_i \leftarrow \text{Generate}(\Theta_{\text{cand}}^i, \bar{t})$
 - 8: **end for**
 - 9: **end for**
 - 10: **for** $i = 2$ to $n - 1$ **do** ▷ Assuming $\{\alpha_n\}$ is monotonic
 - 11: **for each** hyper-parameter candidate α_i in $\{\alpha_n\}$ **do**
 - 12: $D_i \leftarrow \text{DiffCnt}(G_i, G_{i-1}) + \text{DiffCnt}(G_i, G_{i+1})$
 - 13: **end for**
 - 14: **end for**
 - 15: $i^* \leftarrow \underset{i}{\operatorname{argmin}}(D_i)$
 - 16: $\Theta_{\text{out}} \leftarrow \Theta_{\text{cand}}^{i^*}$
 - 17: **return** Θ_{out}
-

Dare-ties performs Ties-Merging [26].

- **MetaGPT** [32] separate the data term and scaling coefficients in the optimization objective, which leads to a task-agnostic closed-form solution for the scaling coefficient.

4.2. Models

We have conducted extensive experiments on the combinations of existing open-source 7B-scale MLLMs. Since most of the top-performing open-source MLLMs are currently based on two language model architectures, Qwen2 [27] and LLaMA [20], we selected representative and outstanding MLLMs derived from each model for our main experiments. Specifically, on Qwen2 architecture, we merge LLaVA-OneVision-Qwen-7B [12] into Qwen2-VL-7B [24], and on LLaMA architecture, we merge LLaVA-v1.5-7B [14] into CogVLM-Chat-7B [25].

We have also conducted experiments on combinations of LLaMA-based MLLMs, including combinations LLaVA-v1.5-7B, CogVLM-Chat-7B, ShareGPT4V-7B [2] and mPLUG-Owl2-LLaMA2-7B [28]. See Appendix A for more details.

Model	MMMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Original Models										
Qwen2-VL(base)	50.11	81.44	75.85	86.00	84.12	51.43	61.80	68.32	559.07	2
LLaVA-OneVision	43.44	77.04	75.44	69.60	78.47	49.57	59.84	60.97	514.37	0
Baselines										
Task Arithmetic	48.44(+1.67)	82.33(+3.09)	75.81(+0.17)	77.90(+0.10)	76.22(-5.08)	50.60(+0.10)	62.26(+1.44)	62.76(-1.89)	536.32(-0.40)	1
Ties-Merging	51.11(+4.34)	<u>82.65(+3.41)</u>	<u>76.29(+0.64)</u>	84.40(+6.60)	79.56(-1.74)	<u>52.56(+2.06)</u>	61.84(+1.02)	66.34(+1.69)	554.75(+18.03)	4
DARE-Linear	43.78(-3.00)	66.06(-13.18)	74.32(-1.33)	72.40(-5.40)	64.65(-16.65)	43.41 (-7.09)	55.13(-5.69)	50.18(-14.47)	469.93(-66.79)	0
DARE-Ties	45.00(-1.78)	54.43(-24.81)	74.07(-1.58)	75.20(-2.60)	78.54(-2.76)	49.61(-0.89)	58.51(-2.31)	58.05(-6.60)	493.41 (-43.31)	0
MetaGPT	50.67(+3.90)	81.21(+1.97)	76.35(+0.70)	85.50(+7.70)	<u>83.63(+2.33)</u>	52.24(+1.74)	61.99(+1.17)	69.16(+4.51)	<u>560.75(+24.03)</u>	5
Our Method										
AdaMMS	51.11(+4.34)	83.36(+4.12)	76.20(+0.55)	85.50(+7.70)	<u>83.41(+2.11)</u>	53.56(+3.06)	<u>62.02(+1.20)</u>	<u>68.40(+3.75)</u>	563.56(+26.84)	8

Table 1. Results on merging LLaVA-OneVision-7B into Qwen2-VL-7B. All the scores have been scaled to 0-100. SUM refers to the sum of scores on all tasks after scaling. Top2 column represents the number of tasks obtained by this method from the top two among all methods. The number in the parenthesis indicates the performance improvement compared with the average score of original models. The results in the original models that are higher than all model merging methods are highlighted in italics.

Model	MMMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Original Models										
CogVLM(base)	34.80	59.23	61.22	56.50	77.57	60.82	59.43	37.09	446.66	2
LLaVA	35.10	66.68	60.52	31.30	46.04	53.42	<i>61.94</i>	54.29	409.29	0
Baselines										
Task Arithmetic	36.20 (+1.25)	<u>65.99</u> (+3.03)	65.85 (+4.98)	51.20 (+7.30)	68.21 (+6.40)	61.92 (+4.80)	58.82 (-1.87)	35.70 (-9.99)	443.89 (+15.91)	4
Ties-Merging	34.00 (-0.95)	57.29 (-5.67)	38.97 (-21.90)	55.00 (+11.10)	59.73 (-2.08)	40.31 (-16.81)	51.97 (-8.72)	24.36 (-21.33)	361.63 (-66.35)	0
DARE-Linear	36.80 (+1.85)	64.08 (+1.12)	<u>65.07 (+4.20)</u>	47.90 (+4.00)	65.35 (+3.54)	60.96 (+3.84)	58.01 (-2.68)	36.12 (-9.57)	434.29 (+6.31)	2
DARE-Ties	33.60 (-1.35)	46.75 (-16.21)	58.41 (-2.46)	26.50 (-17.40)	50.48 (-11.33)	53.15 (-3.97)	49.62 (-11.07)	31.43 (-14.26)	349.94 (-78.04)	0
MetaGPT	34.70 (-0.25)	59.37 (-3.59)	61.29 (+0.42)	56.40 (+12.50)	76.96 (+15.15)	60.84 (+3.72)	<u>59.44 (-1.25)</u>	<u>36.97 (-8.72)</u>	<u>445.97 (+17.99)</u>	5
Our Method										
AdaMMS	34.90 (-0.05)	69.09 (+6.13)	64.12 (+3.25)	<u>55.70 (+11.80)</u>	<u>76.90 (+15.09)</u>	<u>61.11 (+3.99)</u>	60.12 (-0.57)	37.27 (-8.42)	459.21 (+31.23)	7

Table 2. Results on merging LLaVA-v1.5-7B into CogVLM-chat-7B.

4.3. Benchmarks

To evaluate the capabilities of the merged MLLMs, we have conducted experiments on various benchmarks that cover a wide range of vision-language abilities. According to the classification in [13], our benchmarks fall into three categories: (1) comprehensive-evaluation, (2) cognition and reasoning, (3) text-rich VQA. The comprehensive-evaluation tasks consist of MME [5], SeedBench [11] and VizWiz [7]. Cognition and reasoning type include MMMU [30], OK-VQA [16] and GQA [9]. Text-rich VQA type encompass OCRBench [15] and TextVQA [18].

To present the overall performance of the MLLMs in a standardized manner, we apply linearly normalization to the scores across all tasks, scaling them to a range from 0 to 100. Specially, the total score of MME is 2800, which we have divided by 28 for scaling purposes.

4.4. Implementation

To apply our unsupervised hyper-parameter selection method in the searching step, we need to specify the candidates of α (linear interpolation coefficient). We sample the candidates in a subinterval of $[0, 1]$ with a fixed granularity. **Subinterval** We find that merging with $\alpha \geq 0.7$ often results in collapsing language ability of the merged model, therefore we empirically limit the subinterval of α candi-

dates to $[0, 0.6]$ for eliminating unnecessary search.

Granularity We use granularity to determine the interval between two adjacent candidates of α . In our main experiments, we choose the granularity as 0.1 to obtain satisfying performance with acceptable computation cost.

Evaluation Framework We evaluated the benchmarks with LMMs-Eval [31] and VLMEvalKit [3], two open-source evaluation frameworks for MLLMs.

Subset Searching Instead of conducting search across the entire available input data, we strategically utilize a small subset of only 100 inputs during the search phase, reducing the data volume by at least an order of magnitude. Experimental results demonstrate that this maintains performance without compromising effectiveness.

5. Results

As described in Section 4.2, the main results on distinct vision-language benchmarks are conducted with two representative MLLM pairs. Specifically, Table 1 shows the results of merging LLaVA-OneVision-7B’s parameters into Qwen2-VL-7B’s parameters and architecture, and Table 2 shows the results of merging LLaVA-v1.5-7B’s parameters into CogVLM-chat-7B’s parameters and architecture. Results of other model pairs and larger models can be found in Appendix A.

Method	MMMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	ScienceQA	OKVQA	GQA	VizWiz _{val}	SUM
EM-Full	51.11	83.36	76.34	85.50	83.41	85.69	53.56	62.02	68.40	563.70
Emb-Full	50.56	83.36	76.34	85.50	83.41	85.69	53.56	61.44	68.40	562.57
EM-Sample100	51.11	83.36	76.20	85.50	83.41	86.55	53.56	62.02	68.40	562.12
Emb-Sample100	51.11	82.36	76.34	85.50	83.41	85.69	53.56	61.44	68.40	563.56

Table 3. Results on AdaMMS when merging LLaVA-OneVision-7B into Qwen2-VL-7B using exact match (EM-) and sentence embedding (Emb-) to calculate the differences in searching phase, using full test set inputs (-Full) and a sampled subset of 100 inputs (-Sample100).

Model	MMMU _{val}	MME _{sum}	OCRBench
α -0.00	50.11	81.44	86.00
α -0.10	50.56	81.46	85.50
α -0.20	51.11	82.36	85.20
α -0.30	51.22	83.36	84.40
α -0.40	50.67	83.03	80.70
α -0.50	50.00	81.37	76.40
α -0.60	47.00	82.06	71.20
Oracle	51.22(0.30)	83.36(0.30)	85.50(0.10)
AdaMMS	51.11(0.20)	83.36(0.30)	85.50(0.10)

Table 4. Results with α granularity of 0.1 when merging LLaVA-OneVision-7B into Qwen2-VL-7B. The values in parentheses indicate the selected α . Oracle represents the best possible performance (upper bound) for each task, while AdaMMS shows the results achieved by our unsupervised selection method.

Original Models		
LLaVA-OneVision	69.60	69.60
Qwen2-VL	86.00	86.00
Merging-Base	LLaVA-OneVision	Qwen2-VL
Baselines		
Task Arithmetic	68.10	77.90
Ties-Merging	56.10	84.40
DARE-Linear	63.90	72.40
DARE-Ties	64.40	75.20
MetaGPT	38.40	85.50
Linear Interpolation		
α -0.10	70.60	85.50
α -0.20	71.70	85.20
α -0.30	69.90	84.40
α -0.40	67.00	80.70
α -0.50	62.00	76.40
α -0.60	54.40	71.30
Our Method		
AdaMMS	70.60	85.50

Table 5. Results on OCRBench when merging LLaVA-OneVision-7B and Qwen2-VL-7B.

AdaMMS addresses the challenges of merging for heterogeneous MLLMs and outperforms strong baselines. As demonstrated in Table 1 and Table 2, our proposed AdaMMS model merging method achieves the highest cumulative performance scores across both MLLM

pairs, indicating its effectiveness in merging heterogeneous MLLMs. Ranks among the top two performs in 8 out of 9 metrics in Table 1 and 7 metrics in Table 2, demonstrating its consistent ability to adaptively improve performance across most tasks. Moreover, our method stands out as the only approach where the merged model significantly outperforms both pre-merged models, achieving an average gain of +3.36 (total gain of +26.84) over Qwen2-VL and +3.90 over CogVLM across 8 tasks. Given that most baseline methods employ supervised search techniques that incorporate additional information, our unsupervised search approach demonstrates exceptional performance on vision-language benchmarks, as detailed illustrated in Appendix D.

The proposed unsupervised hyper-parameter selection method is able to select a near-optimal α . We evaluate the performance across different coefficient values α , comparing the results obtained through our unsupervised hyper-parameter selection method against those achieved with the optimal α chosen by the actual best results, which serves as the theoretical upper bound. As shown in Table 4, our method consistently performs remarkably close to this upper bound, with a maximum deviation of only 0.5 points. These results demonstrate the capability of our method to accurately identify near-optimal α values, achieving performance levels approaching the theoretical best.

Note that on the OCRBench and TextVQA benchmarks, all model merging methods, including AdaMMS, show a performance drop compared to the original base model. We hypothesize that this is due to the large performance gap between the two original models on these benchmarks. Even though, AdaMMS still outperforms most of the baselines, showing the robustness of our method on various scenarios.

6. Analysis

6.1. Different Factors for Calculating Generation Consistency

We conducted analytical experiments on our generation consistency calculation methods, focusing on two key factors: the choice between using a 100-sample subset versus the complete dataset, and the selection of evaluation metrics. In the searching step of our method, we employed an exact match metric to calculate DiffCnt in Algorithm 1,

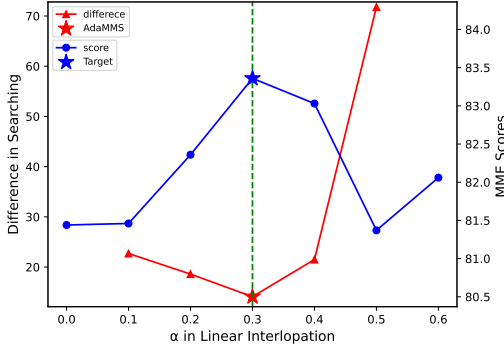


Figure 2. Results on merging LLaVA-v1.5-7B into Qwen2-VL-7B. The α with the best perfo, bb=0 0 461 346rmance are the same as the α with the fewest response differences.

which serves as our generation consistency indicator for model performance prediction. Given that exact match is a binary, rigorous evaluation metric, we explored an alternative, more flexible approach to measure generation consistency. Specifically, we computed the cosine similarity between sentence embeddings generated by all-MiniLM-L6-v2 [17], which was used to calculate DiffCnt. The analysis results are presented in Table 3. Although embeddings theoretically offer more fine-grained semantic representations, our results demonstrate that the embedding-based metric performs comparably to the exact match metric. Furthermore, our experiments confirm that sampling 100 instances achieves results nearly equivalent to the complete dataset.

6.2. Merging with Large Performance Gap

As discussed in Section 5, all model merging methods experience performance drop after the merging on two benchmarks, OCRBench and TextVQA. It shows that merging a model with significant *lower* performance into the base model will decrease the performance on the task. Conversely, in Table 5, the merging from Qwen2-VL-7B to LLaVA-OneVision-7B shows that merging a model with significant *higher* performance into the base model will not necessarily improve the model performance. And in this case, AdaMMS is the only model merging method that resists the performance drop after merging. In general, we observed that original models with similar performance tends to benefit from model merging, while original models with large performance gap do not.

6.3. Asymmetry in the Parameter Space of Heterogeneous Models

In Section 4.4, we discussed that merging with large α often results in collapsing language ability. To validate our choice of the subinterval $[0, 0.6]$ in determining candidates of α , we demonstrate this phenomenon in Figure 3, which shows

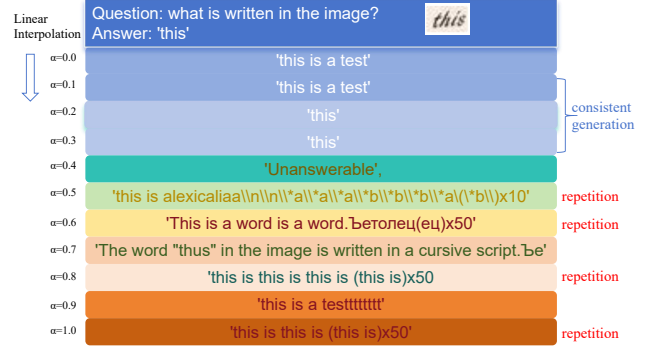


Figure 3. Model responses with the change of α in linear interpolation. Similar colors indicate similar responses.

that the model generates consistently near the parameters of the base model with small α , and collapses gradually with larger α . We attribute the phenomenon to the asymmetry in the parameter space, as the two original models have unequal status that comes from the choice of base architecture.

6.4. Selection of Granularity for α

To validate our choice of the granularity in Section 4.4, we conducted additional experiments with various granularities of α candidates on n MME and OCRBench when merging LLaVA-OneVision-7B into Qwen2-VL-7B. As shown in Appendix G, the result shows that the difference of selected α and model performance do not change significantly with different granularities. This shows that our choice of granularity as 0.1 would result in comparable performance, with less computation cost.

7. Conclusion

In this work, we propose a novel model merging method AdaMMS to address the challenges in merging heterogeneous MLLMs. We first connect the parameters of different MLLMs through a mapping function, enabling merging operations. We then apply linear interpolation to the mapped model weights to adaptively optimize performance across tasks. To optimize the interpolation coefficient without labeled data, we introduce an unsupervised hyperparameter searching method based on our discovery in the parameter space: model performance can be estimated through the generation consistency. We demonstrate that 100 data samples are enough to search for near-optimal coefficients effectively. Extensive experimental results show that AdaMMS outperforms existing model merging methods for MLLMs and successfully addresses the challenges in merging heterogeneous MLLMs. We hope that our work mitigates the limitations of heterogeneous model merging methods and provides valuable insights for future research on unsupervised performance estimation and optimization.

Acknowledgment

This work is supported by the National Key R&D Program of China (2022ZD0160502) and the National Natural Science Foundation of China (No. 62276152).

References

- [1] Chi Chen, Yiyang Du, Zheng Fang, Ziyue Wang, Fuwen Luo, Peng Li, Ming Yan, Ji Zhang, Fei Huang, Maosong Sun, and Yang Liu. Model composition for multimodal large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2024.
- [2] Lin Chen, Jinsong Li, Xiaoyi Dong, Pan Zhang, Conghui He, Jiaqi Wang, Feng Zhao, and Dahua Lin. Sharegpt4v: Improving large multi-modal models with better captions. *arXiv preprint arXiv:2311.12793*, 2023.
- [3] Haodong Duan, Junming Yang, Yuxuan Qiao, Xinyu Fang, Lin Chen, Yuan Liu, Amit Agarwal, Zhe Chen, Mo Li, Yubo Ma, Hailong Sun, Xiangyu Zhao, Junbo Cui, Xiaoyi Dong, Yuhang Zang, Pan Zhang, Jiaqi Wang, Dahua Lin, and Kai Chen. Vlmevalkit: An open-source toolkit for evaluating large multi-modality models, 2024.
- [4] Cl  mentine Fourrier, Nathan Habib, Alina Lozovskaya, Konrad Szafer, and Thomas Wolf. Open llm leaderboard v2. https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard, 2024.
- [5] Chaoyou Fu, Peixian Chen, Yunhang Shen, Yulei Qin, Mengdan Zhang, Xu Lin, Jinrui Yang, Xiawu Zheng, Ke Li, Xing Sun, et al. Mme: A comprehensive evaluation benchmark for multimodal large language models. *arXiv preprint arXiv:2306.13394*, 2023.
- [6] Charles Goddard, Shamane Siriwardhana, Malikeh Ehghaghi, Luke Meyers, Vlad Karpukhin, Brian Benedict, Mark McQuade, and Jacob Solawetz. Arcee’s mergekit: A toolkit for merging large language models. *arXiv preprint arXiv:2403.13257*, 2024.
- [7] Danna Gurari, Qing Li, Abigale J Stangl, Anhong Guo, Chi Lin, Kristen Grauman, Jiebo Luo, and Jeffrey P Bigham. Vizwiz grand challenge: Answering visual questions from blind people. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3608–3617, 2018.
- [8] Wenyi Hong, Weihan Wang, Ming Ding, Wenmeng Yu, Qingsong Lv, Yan Wang, Yean Cheng, Shiyu Huang, Junhui Ji, Zhao Xue, et al. CogVLM2: Visual language models for image and video understanding. *arXiv preprint arXiv:2408.16500*, 2024.
- [9] Drew A Hudson and Christopher D Manning. Gqa: A new dataset for real-world visual reasoning and compositional question answering. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6700–6709, 2019.
- [10] Gabriel Ilharco, Marco Tulio Ribeiro, Mitchell Wortsman, Suchin Gururangan, Ludwig Schmidt, Hannaneh Hajishirzi, and Ali Farhadi. Editing models with task arithmetic. *arXiv preprint arXiv:2212.04089*, 2022.
- [11] Bohao Li, Yuying Ge, Yixiao Ge, Guangzhi Wang, Rui Wang, Ruimao Zhang, and Ying Shan. Seed-bench: Benchmarking multimodal large language models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 13299–13308, 2024.
- [12] Bo Li, Yuanhan Zhang, Dong Guo, Renrui Zhang, Feng Li, Hao Zhang, Kaichen Zhang, Yanwei Li, Ziwei Liu, and Chunyuan Li. Llava-onevision: Easy visual task transfer. *arXiv preprint arXiv:2408.03326*, 2024.
- [13] Jian Li and Weiheng Lu. A survey on benchmarks of multimodal large language models. *arXiv preprint arXiv:2408.08632*, 2024.
- [14] Haotian Liu, Chunyuan Li, Yuheng Li, and Yong Jae Lee. Improved baselines with visual instruction tuning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 26296–26306, 2024.
- [15] Yuliang Liu, Zhang Li, Biao Yang, Chunyuan Li, Xucheng Yin, Cheng-lin Liu, Lianwen Jin, and Xiang Bai. On the hidden mystery of ocr in large multimodal models. *arXiv preprint arXiv:2305.07895*, 2023.
- [16] Kenneth Marino, Mohammad Rastegari, Ali Farhadi, and Roozbeh Mottaghi. Ok-vqa: A visual question answering benchmark requiring external knowledge. In *Proceedings of the IEEE/cvf conference on computer vision and pattern recognition*, pages 3195–3204, 2019.
- [17] Nils Reimers and Iryna Gurevych. Sentence-bert: Sentence embeddings using siamese bert-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 2019.
- [18] Amanpreet Singh, Vivek Natarajan, Meet Shah, Yu Jiang, Xinlei Chen, Dhruv Batra, Devi Parikh, and Marcus Rohrbach. Towards vqa models that can read. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 8317–8326, 2019.
- [19] Yi-Lin Sung, Linjie Li, Kevin Lin, Zhe Gan, Mohit Bansal, and Lijuan Wang. An empirical study of multimodal model merging. *arXiv preprint arXiv:2304.14933*, 2023.
- [20] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timoth  e Lacroix, Baptiste Rozi  re, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models.
- [21] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017.
- [22] Fanqi Wan, Xinting Huang, Deng Cai, Xiaojun Quan, Wei Bi, and Shuming Shi. Knowledge fusion of large language models, 2024.
- [23] Fanqi Wan, Ziyi Yang, Longguang Zhong, Xiaojun Quan, Xinting Huang, and Wei Bi. Knowledge fusion of chat llms: A preliminary technical report, 2024.
- [24] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin

- Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024.
- [25] Weihang Wang, Qingsong Lv, Wenmeng Yu, Wenyi Hong, Ji Qi, Yan Wang, Junhui Ji, Zhuoyi Yang, Lei Zhao, Xixuan Song, et al. Cogvlm: Visual expert for pretrained language models. *arXiv preprint arXiv:2311.03079*, 2023.
- [26] Prateek Yadav, Derek Tam, Leshem Choshen, Colin A Raffel, and Mohit Bansal. TIES-Merging: Resolving interference when merging models. *Advances in Neural Information Processing Systems*, 36, 2024.
- [27] An Yang, Baosong Yang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Zhou, Chengpeng Li, Chengyuan Li, Dayiheng Liu, Fei Huang, Guanting Dong, Haoran Wei, Huan Lin, Jialong Tang, Jialin Wang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Ma, Jianxin Yang, Jin Xu, Jingren Zhou, Jinze Bai, Jinzheng He, Junyang Lin, Kai Dang, Keming Lu, Keqin Chen, Kexin Yang, Mei Li, Mingfeng Xue, Na Ni, Pei Zhang, Peng Wang, Ru Peng, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Zhang, Xipin Wei, Xuancheng Ren, Xuejing Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yaqiong Liu, Zeyu Cui, Zhenru Zhang, Zhifang Guo, and Zhihao Fan. Qwen2 technical report, 2024.
- [28] Qinghao Ye, Haiyang Xu, Jiabo Ye, Ming Yan, Anwen Hu, Haowei Liu, Qi Qian, Ji Zhang, and Fei Huang. mplug-owl2: Revolutionizing multi-modal large language model with modality collaboration. In *2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13040–13051. IEEE, 2024.
- [29] Le Yu, Bowen Yu, Haiyang Yu, Fei Huang, and Yongbin Li. Language models are super mario: Absorbing abilities from homologous models as a free lunch. In *Forty-first International Conference on Machine Learning*, 2024.
- [30] Xiang Yue, Yuansheng Ni, Kai Zhang, Tianyu Zheng, Ruochi Liu, Ge Zhang, Samuel Stevens, Dongfu Jiang, Weiming Ren, Yuxuan Sun, et al. MMMU: A massive multi-discipline multimodal understanding and reasoning benchmark for expert agi. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 9556–9567, 2024.
- [31] Kaichen Zhang, Bo Li, Peiyuan Zhang, Fanyi Pu, Joshua Adrian Cahyono, Kairui Hu, Shuai Liu, Yuanhan Zhang, Jingkang Yang, Chunyuan Li, and Ziwei Liu. Lmms-eval: Reality check on the evaluation of large multimodal models, 2024.
- [32] Yuyan Zhou, Liang Song, Bingning Wang, and Weipeng Chen. MetaGPT: Merging large language models using model exclusive task arithmetic. *arXiv preprint arXiv:2406.11385*, 2024.

A. Results on Additional Model Pairs

We conducted experiments on additional model pairs, summarized in Table 9, which highlights the cumulative performance gains across tasks for six different model pairs. The model pairs include: (1) merging LLaVA-OneVision [12] into Qwen2-VL [24] (Table 1), (2) merging LLaVA-v1.5 [14] into CogVLM [25] (Table 2), (3) merging mPLUG-Owl2 into LLaVA-v1.5, (4) merging LLaVA-v1.5 into mPLUG-Owl2 [28] (Table 11), (5) merging CogVLM into mPLUG-Owl2 (Table 12) and (6) merging mPLUG-Owl2 into CogVLM (Table 13). The performance gain for each task is computed as the difference between the performance of our method (or baselines) and the average performance of the two original models, with positive values indicating an improvement. In Table 9, the SUM column presents the total performance gains across all tasks, where AdaMMS outperforms all baselines, achieving +91.92 performance gain, and consistently ranks among the top two in performance gains across all benchmarks. It is noteworthy that on GQA [9] and VizWiz [7] benchmarks in Table 11 and Table 12, all model merging methods experience a performance drop. We attribute this decline to the significant performance gap between the original models on these benchmarks. In these scenarios, AdaMMS demonstrates the smallest performance decrease among them. In Table 11, Table 12 and Table 13, AdaMMS obtains the second best result in the sum of all benchmarks, with a small gap compared to the best baseline.

To investigate the effect of altering base models on performances, we analyze experiments on merging the same model pair with different base models. For the model pair of mPLUG-OWl2 and CogVLM, results in Table 12 use mPLUG-Owl2 as the base model, and results in Table 13 use CogVLM as the base model. On benchmarks where the original models exhibit a significant performance gap, such as OCRBench [15] and TextVQA [18], model merging methods, including AdaMMS, achieve only marginal performance improvements. In contrast, on benchmarks where the original models have comparable performance, AdaMMS consistently enhances the base model’s performance (with the exception of GQA [9] for the mPLUG-Owl2 architecture), irrespective of the choice of base model. Notably, even when merging a weaker model into a stronger one for a specific task, AdaMMS can sometimes boost the stronger model’s performance. For instance, this effect is observed on SEEDBench [11], OKVQA [16], and GQA [9] in Table 13. These results highlight that our model merging technique can further optimize the performance of a strong model, even when another model demonstrates weaker performance on the same task.

Additionally, to demonstrate the effectiveness of our method on larger models, we conducted experiments on Cambrian and Yi-VL with 34B language model size. Ta-

ble 6 shows that AdaMMS also merges the abilities in larger MLLMs effectively.

Model	OCRBench	MME
Cambrian(base)	58.70	72.50
Yi-VL	29.70	73.65
AVG	44.20	73.08
AdaMMS	59.20	74.07

Table 6. Results on merging Yi-VL into Cambrian.

B. Implementation Details of AdaMMS

The implementation details of AdaMMS are as follows:

Mapping In this step, we identify parameters in the language models that account for additional weights. For CogVLM [25], all weights within the visual experts in the attention mechanism (including the QKV matrix and the FFN of the visual expert) are treated as additional weights. For mPLUG-Owl2 [28], vision representation weights within the Modality-Adaptive Modules (such as the decoupled vision layer-norm and KV matrix) are considered additional weights. For different vision encoders, the vision encoder weights of the base model are retained as the final weights after merging, regardless of the vision encoder in the other model.

Merging During this step, we first merge the weights in the language model of the base model. If the weights are not classified as additional weights in the Mapping step, they are merged using linear interpolation or other baseline merging techniques. For weights categorized as additional weights, we check whether the other model has duplicated the same weights. Based on this, we (1) merge the weights if duplicates exist, or (2) retain the original weights in the base model if no duplicates are found.

Searching In the final step, we randomly select a subset of 100 test inputs to determine the optimal α . For each α candidate, we generate model responses for the selected inputs. To select the best α , we apply the Exact Match metric for the total difference score: for each input, if the merged model’s response with a given α matches the response with adjacent α values, the difference score is 0; otherwise, it is 1. The total difference score is the sum of scores across all inputs in the subset. The α with the lowest total difference score is selected as the final choice. Note that the small subset of 100 inputs is randomly sampled using the method in LMMs-Eval framework [31]. We have repeated the sampling process to ensure that the randomness in sampling does not affect the performance of our method.

C. Evaluation Details

We utilize LMMs-Eval [31] and VLMEvalKit [3], two open-source evaluation frameworks for MLLMs, to as-

sess our models. Specifically, for evaluating MMMU [30], MME [5], SEEDBench [11], OCRBench [15], and TextVQA [18] within the Qwen2-VL [24] architecture, we use the VLMEvalKit framework, while LMMs-Eval is employed for the others. To ensure consistency with the reported results for LLaVA and mPLUG-Owl2 on OK-VQA [16], we adapted the prompt template in the evaluation framework, as detailed in Table 7. Other prompt templates remains the same in the evaluation frameworks.

D. Comparing Supervised and Unsupervised

We compared AdaMMS with baseline merging methods with supervised hyper-parameter selection. Due to the absence of separate test sets, we trained the supervised baseline on either a subset or the entirety of the evaluation set. **This implies that the supervised baseline was in a more favorable position compared to our method, as our method does not have access to the groudtruth labels.** Table 8 shows that AdaMMS still outperforms it, indicating the superiority of our unsupervised method.

E. Intermediate Results in Searching

We present an example of the intermediate results during the selection of α . As shown in Figure 14, AdaMMS effectively identifies a near-optimal α , achieving performance close to the best possible outcome. Specifically, our unsupervised hyper-parameter selection method successfully chooses the optimal α candidate in half of the benchmarks and maintains a deviation of no more than 0.2 from the best α in the remaining cases.

Figure 5 illustrates the relationship between model performance and generation consistency across MMMU, MME, SeedBench, and OCRBench when merging LLaVA-OneVision into Qwen2-VL. The observed trends validate our approach in the search step, where model performance is approximated using generation consistency without relying on labeled data. Notably, for these tasks, the α selected by our method corresponding to the highest generation consistency deviates from the α achieving the best performance by no more than 0.1, showing that our hyper-parameter selection method achieves near-optimal performance.

Framework	Base Model	Prompt
LMMs-Eval	LLaVA	Answer the question using a single word or phrase.
	mPLUG-Owl2	

Table 7. Altered prompt for evaluation on OK-VQA.

F. Supplementary Proof

We provide the following proof as the theoretical justification for relationship between generation consistency correlates and model performance.

Proof. Using the notation in Section 3.3, for an arbitrary task t_i , let $S_{t_i}(\alpha)$ be the ratio of correct answer at position α , and $D_{t_i}(\alpha; \alpha^-)$ be the ratio of the difference in generated responses between position α and its adjacent candidate α^- . Since the difference in $S_{t_i}(\alpha)$ is only influenced by the subset of generated responses where the correctness status changes (i.e., transitions between correct and incorrect), we have $|S_{t_i}(\alpha) - S_{t_i}(\alpha^-)| \leq D_{t_i}(\alpha; \alpha^-)$. For the same reason with α^+ , we can prove $|S_{t_i}(\alpha) - S_{t_i}(\alpha^-)| + |S_{t_i}(\alpha) - S_{t_i}(\alpha^+)| \leq 2D_{t_i}(\alpha; \alpha^-, \alpha^+)$. Therefore, a higher generation consistency with small $D_{t_i}(\alpha; \alpha^-, \alpha^+)$ implies a higher model performance $S_{t_i}(\alpha)$, due to its convexity.

G. Experimental Results in Granularity for α

Figure 4 presents the result of AdaMMS at different granularities of α . The point in stars indicates the best α by our unsupervised parameter selection method. The result shows that these granularities in $\{0.02, 0.05, 0.10\}$ behave similarly in terms of the final performance, indicating the robustness of AdaMMS. Therefore, in practice we choose a larger α so that we have fewer α candidates, which reduces the computation cost.

Model	MMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	Sum	Diff
AdaMMS	34.90	69.09	64.12	55.70	76.90	61.11	60.12	37.27	459.21	+31.23
Ties-Merging	34.00	57.29	38.97	55.00	59.73	40.31	51.97	24.36	361.63	-66.35
Ties-Merging (supervised with 100 eval. samples)	37.20	57.29	63.12	55.90	76.50	61.45	55.81	37.98	445.25	+17.27
Ties-Merging (supervised with all eval. data)	37.20	63.96	65.43	55.90	76.55	61.45	57.99	38.21	456.69	+28.71

Table 8. AdaMMS and Ties-Merging with *supervised* hyper-parameter selection via validation set.

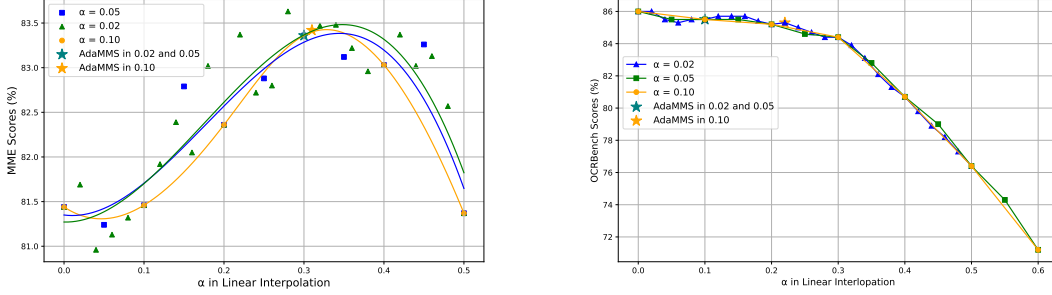


Figure 4. Results on linear interpolation at different granularities of α when merging LLaVA-OneVision-7B into Qwen2-VL-7B-7B. (Left: MME, Right: OCRBench)

Model	MMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Task Arithmetic	<u>13.21</u>	21.53	14.54	-1.80	-3.74	13.88	-2.95	<u>-7.29</u>	47.45	7
Ties-Merging	-3.32	-24.94	-27.34	1.20	-31.59	-23.23	-29.70	-29.20	-168.05	0
DARE-Linear	8.15	-2.35	10.58	-12.3	-19.23	5.41	-12.76	-21.09	-43.53	0
DARE-Ties	-14.83	-60.56	-6.96	-47.50	-47.12	-31.08	-26.32	-32.45	-266.76	0
MetaGPT	1.44	-2.93	-4.02	15.30	<u>0.37</u>	-6.75	-23.69	-16.73	-36.94	2
AdaMMS	17.68	<u>17.48</u>	<u>12.02</u>	<u>13.60</u>	18.43	<u>13.40</u>	1.40	-2.14	91.92	9

Table 9. Results of the performance gain sum among six model pairs reported in our paper, as described in Appendix A. The performance gain for each task is computed as the difference between the performance of our method (or baselines) and the average performance of the two original models, with positive values indicating an improvement.

Model	Unsupervised	MMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Original Models											
LLaVA(base)		35.10	66.68	60.52	31.30	46.04	53.42	61.94	54.29	409.29	
mPLUG-Owl2		34.90	62.80	59.41	34.10	55.13	60.98	56.11	32.07	395.50	
Baselines											
Task Arithmetic	×	36.00 (+1.00)	67.00 (+2.26)	61.45 (+1.48)	30.40 (-2.30)	45.75 (-4.84)	56.79 (-0.41)	59.68 (+0.66)	56.49 (+13.31)	413.56 (+11.17)	5
Ties-Merging	×	33.60 (-1.40)	62.14 (-2.60)	60.32 (+0.35)	30.10 (-2.60)	42.85 (-7.73)	52.46 (-4.74)	58.37 (-0.66)	51.30 (+8.12)	391.14 (-11.25)	0
DARE-Linear	×	36.00 (+1.00)	67.00 (+2.26)	61.41 (+1.44)	30.70 (-2.00)	45.84 (-4.74)	57.06 (-0.14)	59.56 (+0.54)	55.90 (+12.72)	413.47 (+11.08)	2
DARE-Ties	×	31.70 (-3.30)	59.81 (-4.93)	60.06 (+0.09)	29.50 (-3.20)	41.90 (-8.69)	46.00 (-11.20)	57.51 (-1.52)	53.27 (+10.09)	379.75 (-22.64)	0
MetaGPT	✓	35.30 (+0.30)	67.62 (+2.88)	<u>61.46 (+1.49)</u>	<u>30.60 (-2.10)</u>	45.80 (-4.79)	56.54 (-0.66)	59.41 (+0.38)	56.66 (+13.48)	413.39 (+11.00)	4
Our Method											
AdaMMS	✓	38.30 (+3.30)	<u>67.01 (+2.27)</u>	61.82 (+1.85)	31.00 (-1.70)	46.49 (-4.09)	55.60 (-1.60)	61.81 (+2.79)	54.64 (+11.46)	416.67 (+14.28)	7

Table 10. Results on merging mPLUG-Owl2-7B into LLaVA-v1.5-7B.

Model	Unsupervised	MMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Original Models											
mPLUG-Owl2(base)		34.90	62.80	59.41	34.10	55.13	60.98	56.11	32.07	395.50	
LLaVA		35.10	66.68	60.52	31.30	46.04	53.42	61.94	54.29	409.29	
Baselines											
Task Arithmetic	×	<u>36.90(+1.90)</u>	63.17(-1.57)	60.44(+0.47)	33.00(+0.30)	55.40(+4.81)	63.87(+6.67)	56.97(-2.06)	33.70(-9.48)	403.45(+1.05)	4
Ties-Merging	×	<u>36.90(+1.90)</u>	64.20(-0.54)	60.13(+0.16)	34.40(+1.70)	54.50(+3.91)	62.92(+5.72)	57.55(-1.48)	33.18(-10.00)	403.78(+1.38)	4
DARE-Linear	×	<u>36.20(+1.20)</u>	62.99(-1.75)	<u>60.41(+0.44)</u>	32.60(-0.10)	55.15(+4.56)	<u>63.47(+6.27)</u>	56.73(-2.30)	33.35(-9.83)	400.90(-1.50)	2
DARE-Ties	×	<u>35.30(+0.30)</u>	60.37(-4.37)	58.36(-1.61)	32.00(-0.70)	51.65(+1.06)	58.08(+0.88)	55.57(-3.46)	31.03(-12.15)	382.36(-20.04)	0
MetaGPT	✓	36.00(+1.00)	<u>64.24(-0.50)</u>	60.23(+0.26)	<u>33.90(+1.20)</u>	<u>55.83(+5.24)</u>	62.88(+5.68)	56.53(-2.50)	33.35(-9.83)	402.96(+0.56)	3
Our Method											
AdaMMS	✓	37.60(+2.60)	64.61(-0.13)	60.02(+0.05)	32.20(-0.50)	55.84(+5.25)	63.13(+5.93)	<u>56.98(-2.05)</u>	<u>33.39(-9.79)</u>	<u>403.77(+1.37)</u>	6

Table 11. Results on merging LLaVA-v1.5-7B into mPLUG-Owl2-7B.

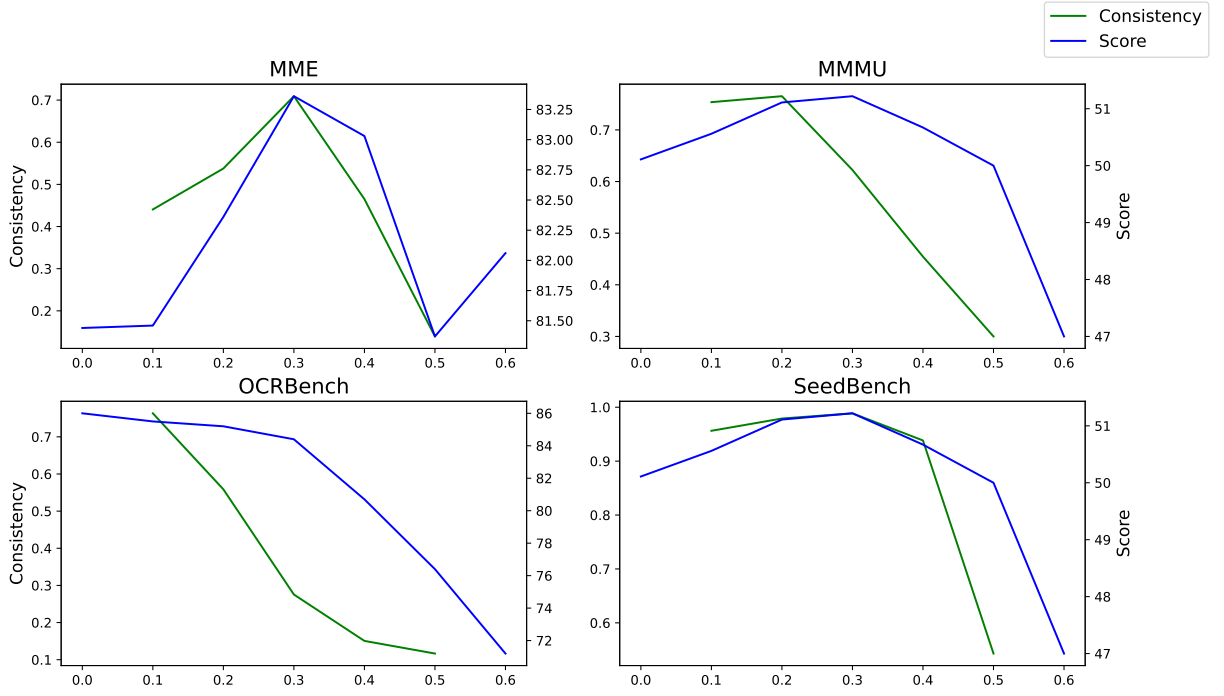


Figure 5. Generation consistency and model performance (score) for MME, MMMU, OCRBench and SeedBench when merging LLaVA-OneVision-7B into Qwen2-VL-7B. Generation consistency is calculated as the reciprocal of the sum of different responses from models with adjacent α candidates. The horizontal axis is the α of the linear interpolation.

Model	Unsupervised	MMMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Original Models											
mPLUG-Owl2(base)		34.90	62.80	59.41	34.10	55.13	60.98	56.11	32.07	395.50	
CogVLM		34.80	59.23	61.22	56.50	77.57	60.82	59.43	37.09	446.66	
Baselines											
Task Arithmetic	×	38.80(+3.95)	64.65(+3.63)	60.85(+0.53)	31.50(-13.80)	56.99(-9.36)	60.93(+0.03)	54.44(-3.33)	32.76(-1.82)	400.92(-20.16)	8
Ties-Merging	×	27.9(-6.95)	48.96(-12.06)	52.32(-8.00)	24.30(-21.00)	42.10(-24.25)	54.15(-6.75)	43.02(-14.75)	27.56(-7.02)	320.31(-100.77)	0
DARE-Linear	×	37.60(+2.75)	62.44(+1.42)	59.81(-0.51)	30.90(-14.40)	56.41(-9.94)	61.07(+0.17)	54.11(-3.66)	32.42(-2.16)	394.76(-26.32)	1
DARE-Ties	×	32.00(-2.85)	57.90(-3.12)	57.62(-2.70)	24.10(-21.20)	43.84(-22.51)	51.56(-9.34)	52.04(-5.73)	25.67(-8.91)	344.73(-76.35)	0
MetaGPT	✓	31.30(-3.55)	56.81(-4.21)	50.81(-9.51)	29.30(-16.00)	37.96(-28.39)	43.02(-17.88)	34.12(-23.65)	15.84(-18.74)	299.16(-121.92)	0
Our Method											
AdaMMS	✓	39.10(+4.25)	64.65(+3.63)	60.16(-0.16)	30.60(-14.70)	55.88(-10.47)	62.11(+1.21)	55.61(-2.16)	32.69(+1.89)	400.80(-20.28)	9

Table 12. Results on merging CogVLM-7B into mPLUG-Owl2-7B.

Model	Unsupervised	MMMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM	Top2
Original Models											
CogVLM(base)		34.80	59.23	61.22	56.50	77.57	60.82	59.43	37.09	446.66	
mPLUG-OWL2		34.90	62.80	59.41	34.10	55.13	60.98	56.11	32.07	395.50	
Baselines											
Task Arithmetic	×	38.30(+3.45)	72.11(+11.09)	67.24(+6.92)	51.90(+6.60)	70.68(+4.33)	63.59(+2.69)	59.98(+2.21)	37.16(+2.58)	460.96(+39.88)	7
Ties-Merging	×	34.60(-0.25)	53.54(-7.48)	61.73(+1.41)	50.70(+5.40)	66.65(+0.30)	58.19(-2.71)	52.66(-5.11)	33.92(-0.66)	411.99(-9.09)	0
DARE-Linear	×	39.20(+4.35)	68.80(+7.78)	66.66(+6.34)	50.90(+5.60)	70.35(+4.00)	63.26(+2.36)	58.80(+1.03)	36.80(+2.22)	454.77(+33.69)	3
DARE-Ties	×	29.00(-5.85)	53.89(-7.12)	61.61(+1.30)	42.90(-2.40)	63.46(-2.89)	54.34(-6.56)	55.54(-2.23)	33.96(-0.62)	394.70(-26.38)	0
MetaGPT	✓	34.90(+0.05)	61.54(+0.52)	62.93(+2.62)	57.30(+12.00)	77.18(+10.83)	61.55(+0.65)	59.93(+2.16)	37.15(+2.57)	452.48(+31.40)	2
Our Method											
AdaMMS	✓	38.10(+3.25)	62.48(+1.46)	66.79(+6.48)	56.30(+11.00)	76.89(+10.54)	61.71(+0.81)	59.96(+2.19)	37.33(+2.75)	459.56(+38.48)	6

Table 13. Results on merging mPLUG-Owl2-7B into CogVLM-7B.

Model	MMMU _{val}	MME _{sum}	SeedBench _{all}	OCRBench	TextVQA _{val}	OKVQA	GQA	VizWiz _{val}	SUM
Original Models									
Qwen2-VL(base)	50.11	81.44	75.85	86.00	84.12	51.43	61.80	68.32	559.07
LLaVA-OneVision	43.44	77.04	75.44	69.60	78.47	49.57	59.84	60.97	514.37
AVG	46.78	79.24	75.65	77.80	81.30	50.50	60.82	64.65	536.72
Linear Interpolation									
α -0.1	50.56	81.46	76.20	85.50	83.41	53.56	62.02	68.40	561.11
α -0.2	51.11	82.36	76.23	85.20	81.74	54.76	62.05	67.12	560.57
α -0.3	51.22	83.36	76.34	84.40	78.43	52.03	61.44	63.91	551.13
α -0.4	50.67	83.03	76.06	80.70	71.66	49.83	60.09	58.43	530.47
α -0.5	50.00	81.37	75.63	76.40	59.13	44.96	55.53	52.60	495.62
α -0.6	47.00	82.06	74.76	71.30	39.37	40.31	54.11	46.39	455.30
Our Method									
AdaMMS	51.11	83.36	76.20	85.50	83.41	53.56	62.02	68.40	563.56
Selected α	0.2	0.3	0.1	0.1	0.1	0.1	0.1	0.1	-
Distance with the best α	0.1	0	0.2	0	0	0.1	0.1	0	-

Table 14. Intermediate results on different α candidates in the linear interpolation of AdaMMS, and the α selected by our unsupervised hyper-parameter selection method on merging LLaVA-OneVision-7B into Qwen2-VL-7B. AVG indicates the average performance of the two original models.